

Vetor Dinâmico em C: Análise de Complexidade

Introdução

Este documento apresenta uma análise da complexidade temporal e espacial da implementação de um vetor dinâmico em C desenvolvida neste projeto.

O objetivo é justificar formalmente a complexidade de cada operação implementada, explicando o raciocínio utilizado durante a análise. Além de servir como documentação complementar ao código-fonte, este material registra o processo de aprendizagem realizado durante o desenvolvimento do projeto.

Ao longo do documento, utilizaremos a seguinte notação:

- n : número de elementos armazenados no vetor;
- capacidade: quantidade de posições atualmente alocadas para o vetor.

1. Inserção sem realocação

Considere o trecho responsável por inserir um elemento quando ainda existe espaço disponível no vetor.

```
vetor[posicao] = numero;  
  
posicao++;
```

Nesta situação são executadas apenas operações elementares:

- acesso a uma posição do vetor;
- atribuição de um valor;
- incremento da variável `posicao`

Essas operações possuem custo constante e independem da quantidade de elementos armazenados. Logo, neste caso, a inserção é executada em $O(1)$.

2. Inserção com realocação

Quando a capacidade do vetor é atingida, torna-se necessário realizar uma realocação de memória por meio da função `realloc()`. Nesse momento, podem ocorrer dois cenários:

Caso 2.1 - Existe espaço contíguo disponível

Se houver espaço livre imediatamente após o bloco atualmente ocupado pelo vetor, o alocador poderá expandir esse bloco sem mover seus elementos para outro endereço de memória. Nesse caso, nenhuma cópia é necessária e a inserção continua apresentando custo constante $O(1)$.

Caso 2.2 - Não existe espaço contíguo disponível

Se não houver espaço contíguo suficiente, o alocador deverá:

1. reservar um novo bloco de memória;
2. copiar todos os n elementos para esse novo bloco;
3. liberar o bloco antigo;
4. inserir o novo elemento.

A cópia dos elementos possui custo $O(n)$, enquanto a inserção do novo elemento custa $O(1)$. Assim, nesse caso, a inserção apresenta custo $O(n) + O(1) = O(n)$.

3. Inserção amortizada

Até este ponto, foi analisado o custo de uma inserção individual. Observou-se que:

- quando não há necessidade de realocação, a inserção possui $O(1)$;
- quando ocorre uma realocação, a inserção pode apresentar custo $O(1)$ ou $O(n)$, dependendo da existência de espaço contíguo disponível.

Entretanto, em estruturas de dados como vetores dinâmicos, também é interessante analisar o custo **médio** de uma sequência de inserções. Essa abordagem é denominada **análise amortizada**.

Neste projeto, sempre que ocorre uma realocação, a capacidade do vetor é dobrada. Assim, as cópias dos elementos acontecem apenas nos instantes em que a capacidade é insuficiente, ocorrendo nas seguintes quantidades:

$$1, 2, 4, 8, \dots, n \quad (n = 2^m)$$

O número total de elementos copiados após n inserções é dado pela soma

$$1 + 2 + 4 + 8 + \dots + 2^m$$

Como a sequência forma uma progressão geométrica, temos

$$1 + 2 + 4 + 8 + \dots + 2^m = 2^{m+1} - 1 = \mathbf{2n - 1}$$

Além das cópias realizadas durante as realocações, cada elemento também é inserido exatamente uma vez. Portanto, o número total de operações executadas ao longo das n inserções é

$$n + (2n - 1) = \mathbf{3n - 1}$$

Como foram realizadas n inserções, o custo médio por inserção é

$$\frac{3n - 1}{n} = 3 - \frac{1}{n}$$

Como $3 - \frac{1}{n} < 3$ para todo $n \geq 1$, temos que o custo médio de cada inserção é limitado por uma constante. Sendo assim, concluímos que a inserção amortizada possui custo $O(1)$.

A análise amortizada não descreve um novo caso de inserção, mas sim uma forma diferente de analisar as operações. Em vez de considerar apenas uma inserção isolada, considera-se uma sequência de inserções, distribuindo o custo das realocações entre todas elas. Dessa forma, embora algumas inserções possam custar $O(n)$, o custo médio por inserção permanece constante.

4. Impressão dos elementos

A função responsável por imprimir os elementos do vetor percorre todas as posições ocupadas exatamente uma vez. O trecho principal da função é dado por:

```
for (i = 0 ; i < posicao - 1 ; i++) {  
  
    printf('%d ', vetor[i]);  
  
}  
  
printf('%d\n', vetor[i]);
```

Para analisar sua complexidade, consideramos as principais operações executadas pelo algoritmo.

Durante a execução do laço de repetição, ocorrem:

- n comparações da condição do laço;
- $n - 1$ incrementos da variável de iteração;
- $n - 1$ chamadas à função `printf()` para a impressão dos primeiros elementos.

Após o término do laço, é realizada mais uma chamada à função `printf()`, responsável por imprimir o último elemento do vetor. Sendo assim, temos um total de $n + (n - 1) + (n - 1) + 1 = 3n - 1$ operações, e, portanto, concluímos que a impressão dos elementos possui complexidade $O(n)$.

5. Soma dos elementos

A função responsável por calcular a soma dos elementos também percorre todas as posições ocupadas pelo vetor exatamente uma vez. O trecho principal da função é dado por:

```
for (int i = 0 ; i < posicao ; i++) {  
  
    soma += vetor[i];  
  
}
```

Para analisar sua complexidade, vamos considerar as principais operações executadas pelo algoritmo.

Durante a execução do laço de repetição, ocorrem:

- $n + 1$ comparações da condição do laço;
- n incrementos da variável de iteração;
- n operações de adição com a variável `soma`.

Dessa forma, temos um total de $(n + 1) + n + n = 3n + 1$ operações, permitindo concluir que a soma dos elementos apresenta complexidade $O(n)$.

6. Leitura da entrada

Para o algoritmo armazenar os n números no vetor, foram necessárias n chamadas à função `scanf()` para a leitura desses números e uma chamada para ler o valor `-1` ou `EOF`. Logo, temos um total de $n + 1$ leituras, e, portanto, concluimos que a leitura da entrada possui complexidade $O(n)$.

7. Complexidade espacial

O algoritmo utiliza memória para:

- variáveis locais;
- vetor dinâmico.

As variáveis locais ocupam quantidade constante de memória, e, portanto, temos que a quantidade de memória utilizada por elas possui complexidade $O(1)$.

O vetor possui capacidade igual à menor potência de 2 maior do que ou igual ao número de elementos armazenados. Ou seja:

$$\text{capacidade} = 2^{\lceil \log_2 n \rceil}$$

Assim, memória utilizada pelo vetor é dada por $2^{\lceil \log_2 n \rceil} \cdot \text{sizeof}(\text{int})$. Como $2^{\lceil \log_2 n \rceil} < 2n$ para todo $n \geq 1$, conclui-se que a quantidade de memória ocupada pelo vetor possui complexidade $O(n)$.

Conclusão

A implementação apresentada combina eficiência temporal e simplicidade estrutural. A utilização de crescimento exponencial reduz drasticamente o número de realocações necessárias, permitindo que a operação de inserção apresente custo amortizado constante, apesar das eventuais realocações custarem $O(n)$ no pior caso.

As demais operações de leitura, impressão e soma percorrem todos os elementos uma única vez, apresentando complexidade linear.

Por fim, a memória utilizada cresce proporcionalmente ao número de elementos armazenados, caracterizando complexidade espacial linear.

Observação: Este documento foi elaborado durante o desenvolvimento do projeto com o objetivo de consolidar o entendimento sobre análise de algoritmos, crescimento exponencial de vetores dinâmicos e análise amortizada. Eventuais melhorias ou refinamentos poderão ser incorporados em versões futuras, acompanhando a evolução do autor.